FUNDAMENTOS DE BASES DE DATOS

Memoria de la Práctica 2: Acceso a BD desde C con ODBC

Autores: Mario Rodriguez y Santiago Pita

Curso Académico: 2026–2027

1 Introducción

En esta práctica se ha implementado una aplicación en C que interactúa con la base de datos `classicmodels` mediante la librería **ODBC**. Siguiendo el enunciado, todas las consultas que reciben parámetros por parte del usuario se han construido utilizando consultas preparadas (*prepared statements* con marcadores ?) y vinculación de variables mediante `SQLBindParameter`, garantizando así la seguridad frente a inyecciones SQL y una correcta gestión de tipos.

A continuación se detalla, para cada submenú del programa (*Products*, *Orders* y *Customers*), la consulta SQL empleada y la lógica de implementación desarrollada en C.

---

2 Menú Products

2.1 Stock

QUERY:

```sql
SELECT quantityinstock
FROM products
WHERE productcode = ?;
```

LÓGICA: Para esta función se ha utilizado la estructura base del `example 3` de ODBC (*prepared statements*), ya que necesitamos solicitar al usuario el código del producto (`productcode`) del cual desea consultar las unidades disponibles. En la consulta SQL marcamos ese parámetro con ? y posteriormente lo vinculamos a nuestra variable en C mediante la función `SQLBindParameter`. Finalmente, ejecutamos la consulta con `SQLExecute`, vinculamos la columna resultante con `SQLBindCol` e imprimimos por pantalla el número de unidades en stock (`quantityinstock`) haciendo uso de `SQLFetch`.

2.2 Find

QUERY:

```
SELECT productcode, productname
FROM products
WHERE productname LIKE '%' || ? || '%'
ORDER BY productcode;
```

LÓGICA: En este apartado, partimos de la base del `example 4` de ODBC porque necesitamos pedir al usuario una cadena de texto con el nombre (o parte del nombre) del producto a buscar. En la query utilizamos el operador `LIKE` concatenando los comodines `'%'` con el parámetro `?` mediante el operador `||`, lo que permite buscar cualquier producto cuyo `productname` contenga la subcadena introducida. Tras vincular la cadena del usuario al marcador `?` con `SQLBindParameter`, recorremos los resultados con un bucle `while (SQL_SUCCEEDED(SQLFetch(stmt)))` para imprimir en cada línea el código (`productcode`) y el nombre (`productname`) de todos los productos encontrados, ya ordenados por código de producto mediante ORDER BY.

---

3 Menú Orders

3.1 Open

QUERY:

```
SELECT o.ordernumber
FROM orders o
WHERE o.shippeddate IS NULL
ORDER BY o.ordernumber;
```

LÓGICA: Se ha usado la función base del `example 3` de ODBC; sin embargo, quitando el bucle `while` de lectura por teclado ya que aquí no pedimos ningún valor al usuario. Simplemente ejecutamos directamente la consulta donde seleccionamos aquellos pedidos cuya fecha de entrega (`shippeddate`) sigue siendo `NULL`, es decir, que todavía no se han enviado, y mostramos los identificadores ordenados por `ordernumber`.

3.2 Range

QUERY:

```
SELECT ordernumber, orderdate, shippeddate
FROM orders
WHERE orderdate >= ? AND orderdate <= ?
ORDER BY ordernumber;
```

LÓGICA: Se ha usado la función base del `example 4` de ODBC ya que necesitamos pedir dos fechas al usuario (inicio y fin del intervalo en formato `YYYY-MM-DD`). En la query marcamos esas dos variables con `?` y luego las vinculamos a las variables correspondientes en C con dos llamadas a `SQLBindParameter` (en las posiciones 1 y 2). A continuación ejecuta-

mos la sentencia y recorremos las filas devueltas imprimiendo `ordernumber`, `orderdate` y `shippeddate` ordenados por número de pedido.

3.3 Detail

QUERY:

```
SELECT o.orderdate, o.status, od.quantityordered, od.priceeach,
    od.productcode
FROM orders o
JOIN orderdetails od ON o.ordernumber = od.ordernumber
WHERE od.ordernumber = ?
ORDER BY od.orderlinenumber;
```

LÓGICA: Aquí, al igual que en *Range*, usamos la base del `example 4` ya que necesitamos pedir el `ordernumber` al usuario y después de eso lo vinculamos a nuestra variable con `SQLBindParameter`. En la query hacemos un `JOIN` entre `orders` y `orderdetails` para conseguir un seguimiento lineal con toda la información que queremos ordenada por `orderlinenumber`. Sin embargo, para imprimirla hemos hecho un condicional `if` controlando la primera iteración para que imprima la cabecera con la información esencial del pedido (`orderdate`, `status` y el coste total) una sola vez, y luego en el bucle se imprime toda la información de cada uno de los productos de la orden (`productcode`, `quantityordered` y `priceeach`) sin que se repita la información general de cabecera.

---

4 Menú Customers

4.1 Find

QUERY:

```
SELECT customers.customername, customers.contactfirstname,
       customers.contactlastname, customers.customernumber
FROM customers
WHERE customers.contactfirstname LIKE '%' || ? || '%'
   OR customers.contactlastname LIKE '%' || ? || '%'
ORDER BY customers.customernumber;
```

LÓGICA: Se ha empleado la base del `example 4` de ODBC puesto que solicitamos al usuario una cadena de búsqueda para el nombre o apellido del contacto. En la consulta SQL comprobamos si la cadena introducida está contenida tanto en `contactfirstname` como en `contactlastname` usando el operador `LIKE` junto con la concatenación de comodines `'%' || ? || '%'`. Como la query cuenta con dos marcadores ?, vinculamos la misma variable de entrada del usuario a ambos parámetros mediante dos llamadas a `SQLBindParameter` (vinculando en los parámetros 1 y 2). Finalmente, recorremos las filas obtenidas mostrando `customername`, `contactfirstname`, `contactlastname` y `customernumber`, ordenados por el identificador del cliente mediante ORDER BY.

4.2 List Products

QUERY:

```
SELECT products.productname, SUM(orderdetails.quantityordered)
FROM orders
JOIN orderdetails ON orderdetails.ordernumber = orders.ordernumber
JOIN products ON products.productcode = orderdetails.productcode
WHERE orders.customernumber = ?
GROUP BY products.productcode, products.productname
ORDER BY products.productcode;
```

LÓGICA: En esta función usamos la estructura del `example 4`, aunque también sirve de apoyo el `example 3` de ODBC porque necesitamos pedir por teclado el identificador del cliente (`customernumber`) y vincularlo al parámetro `?` con `SQLBindParameter`. En la query unimos mediante `JOIN` las tablas `orders`, `orderdetails` y `products` para relacionar los pedidos de ese cliente con los productos concretos. Como un mismo producto ha podido ser solicitado en varios pedidos distintos, utilizamos `GROUP BY products.productcode, products.productname` junto con la función de agregación `SUM(orderdetails.quantityordered)` para calcular el total de unidades solicitadas por producto en una única línea, ordenando el listado final por `productcode`.

4.3 Balance

QUERY:

```
SELECT
    COALESCE((SELECT SUM(amount)
              FROM payments
              WHERE customernumber = c.customernumber), 0) -
    COALESCE((SELECT SUM(od.quantityordered * od.priceeach)
              FROM orders o
              JOIN orderdetails od ON o.ordernumber = od.ordernumber
              WHERE o.customernumber = c.customernumber), 0)
FROM customers c
WHERE c.customernumber = ?;
```

LÓGICA: En esta función la query sí que era un poco más compleja que las anteriores ya que teníamos que hacer la resta de dos subconsultas sobre tablas distintas: la suma de los pagos realizados por el cliente (`payments`) menos el valor de la suma de todos los productos comprados en sus pedidos (`orders` y `orderdetails`), lo que nos daba el balance o saldo real de ese cliente. Además, se ha utilizado `COALESCE(..., 0)` en ambas subconsultas para que, en caso de que un cliente no tenga pagos o pedidos registrados (valor `NULL`), se tome como `0` y la resta no devuelva `NULL`. Quitando eso, el programa en C es bastante similar a los anteriores: vinculamos la `?` del `customernumber` a nuestra variable con `SQLBindParameter` e imprimimos el resultado normalmente.